

## **NYC Taxi Tip Analysis through Machine Learning**

When it comes to generating revenue for taxi companies and the drivers that work under them, maximizing the amount of tip they receive per ride is an important factor. Our project aims to help with that. In this paper, we analyze data from the New York City Taxi and Limousine Commission (TLC) (starting from January 2025) and use classification and regression to predict the occurrence of a tip, as well as the percentage tip for a taxi trip in NYC, using other trip variables from our given data. We use end time, trip distance, average travel speed, passenger count, payment type, and fare amount in order to define a relationship between these data points and the tip occurrence and tip percentage. Then, we use this relationship to predict tip occurrence/percentages for other trips. Our regression approach will use gradient-boosted trees (e.g., XGBoost or HistGradientBoostingRegressor) as our main ML model, alongside a simple neural network. Similarly, our classification approach will use SVCs alongside neural networks. This research also aims to figure out which variables correspond to a higher tip percentage, providing very useful insights for taxi drivers so that they can increase their earnings and vendors can identify trends in customer satisfaction, which possibly could translate to later on achieving greater customer loyalty.

### **1. Introduction**

Tipping is an integral, yet difficult to predict, aspect of many everyday transactions, including taxi rides, and it plays an important role in compensation for drivers. Despite this, there is significant variation in tipping behavior across trips, and can be strongly influenced by social and situational factors that are not always obvious or easy to quantify. For example, the mood of the passenger likely plays a very significant role in determining how much they decide to tip, but such data is difficult to obtain. However, a publicly available dataset provided by New York City's Taxi & Limousine Commission provides a very useful starting point for understanding some of the factors that may be driving tipping percentage. This dataset contains over three million observations in January 2025 alone, each one representing a single trip taken in a Yellow Cab in New York City. We use this dataset in order to answer two key questions in our research. Firstly, a classification task: determining whether or not a passenger decides to leave a tip. Secondly, a regression task: determining what percentage tip a passenger decides to leave. In completing both of these tasks, we analyze several models, ranging from simple linear regression to complex nonlinear approaches such as neural networks. Overall, in this project, we aim to better understand which trip characteristics are most informative for predicting tipping behavior, and to evaluate how effective various models are at classification and regression using this real-world dataset.

### **2. Literature Review**

The New York City Taxi & Limousine Commission (TLC) trip data has become a standard benchmark for studying urban mobility, demand, and pricing. For each ride, the dataset includes pickup and dropoff times and locations, trip distance, fare, payment type, and, for card payments, recorded tips (New York City Taxi and Limousine Commission, 2025).

A large body of prior work uses this dataset but focuses on demand or fares rather than tipping. For example, Wang (2024) employs a hybrid VMD–SSA–LSTM model to forecast yellow taxi demand across different regions of New York City, demonstrating that deep learning can effectively capture short-term temporal patterns. Rathore et al. (2024) study taxi price prediction with a hybrid approach that combines clustering and ordinal regression, converting continuous fares into ordered price categories to improve interpretability. These studies show that the TLC data supports sophisticated models and that features such as time of day, distance, and zone strongly influence demand and fares. However, they do not directly address trip-level tip behavior.

The most relevant prior work on tipping is the UCSD report *Predicting Taxi Tip-Rates in NYC* by Jain et al. (n.d.). Using 2013 yellow taxi data, they define the target as tip percentage and perform exploratory analysis over location, distance, duration, and time-of-day features. They exclude cash payments, since tips on cash rides are typically logged as zero, and propose a spatial model that divides the city into a grid and predicts a trip’s tip rate using the average tip percentage in the corresponding cell, with a fallback to the global mean. This highlights systematic spatial variation in tipping but relies on hand-crafted binning rather than flexible machine-learning models. A more recent GitHub project, *Predicting Taxi Gratuities in New York City*, by Boateng (n.d.), uses 2017 yellow taxi data to classify riders as “generous tippers” (tip  $\geq 20\%$ ) or not. After cleaning and feature engineering, the author trains random forest and XGBoost models; the final random forest achieves high accuracy, and feature-importance analysis identifies trip duration, distance, and cost as key predictors.

Our project sits at the intersection of these strands. Like Jain et al. (n.d.), we use NYC TLC yellow taxi data and focus on tip vs. no tip and tip percentage as the response. Like Boateng (n.d.), we draw on machine learning models that have been successful on TLC data, but we apply them in a classification and regression setting rather than just binary classification. From the broader taxi literature (e.g., Rathore et al., 2024; Wang, 2024), we take the idea that temporal features and basic trip characteristics are informative predictors, but we do not directly use specialized LSTM-based demand models or hybrid clustering–ordinal-regression pricing models. In summary, most prior work either predicts demand or fares at an aggregate level, or predicts tips using spatial averaging or binary “high-tip vs. low-tip” classification. Our contribution is to use recent (2025-onward) NYC yellow taxi data to perform a classification and regression analysis of tip presence and tip percentage with modern neural networks, targeting a more fine-grained, continuous notion of tipping behavior.

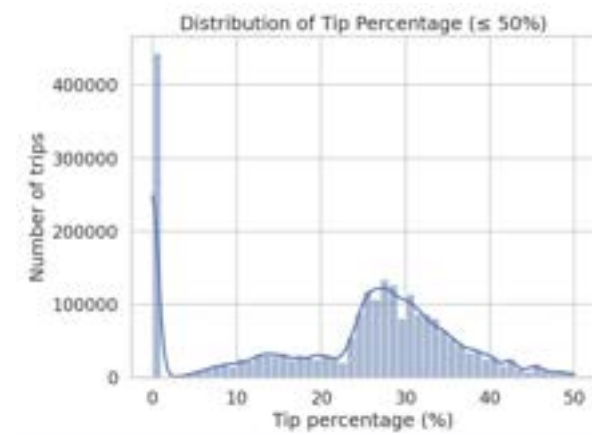
### 3. Exploration of Data

We base our analysis on the January 2025 NYC Yellow Taxi trip data released by the TLC. The raw parquet file contains 3,475,226 trips and 20 variables, including pickup and dropoff timestamps, trip distance, passenger count, pickup and dropoff location IDs, payment type, fare and tip amounts, and several surcharges. As a first cleaning step, we drop rows with missing values, leaving roughly 2.94 million complete records. We then remove trips with non-positive distance or fare and negative tip amounts. To reduce the influence of extreme outliers in the main cost variables, we apply a  $1.5 \times \text{IQR}$  rule separately to `trip_distance` and `fare_amount` and retain only trips whose distance and fare fall within the corresponding lower and upper bounds. Finally, to limit the impact of extreme tipping behavior, we remove trips with tip percentage greater than or equal to 50%. After these filters, the cleaned dataset contains 2,348,490 trips—about 67.6% of the original data—so approximately 32.4% of rows are removed.

On this cleaned dataset, we engineer several features used in both exploration and modeling. Trip duration in hours is computed as  $\text{trip\_hours} = (\text{dropoff\_time} - \text{pickup\_time})$ , average speed as  $\text{trip\_speed} = \text{trip\_distance} / \text{trip\_hours}$ , and tip percentage as  $\text{tip\_percent} = 100 \times \text{tip\_amount} / \text{fare\_amount}$ . We also extract the dropoff hour of day (`dropoff_hour`) and its one-hot encoding, and create a binary indicator `tipped` that is true when the recorded tip percentage is non-zero. These engineered features allow us to examine both continuous variation in tip percentage and the simpler distinction between tipped and non-tipped rides.

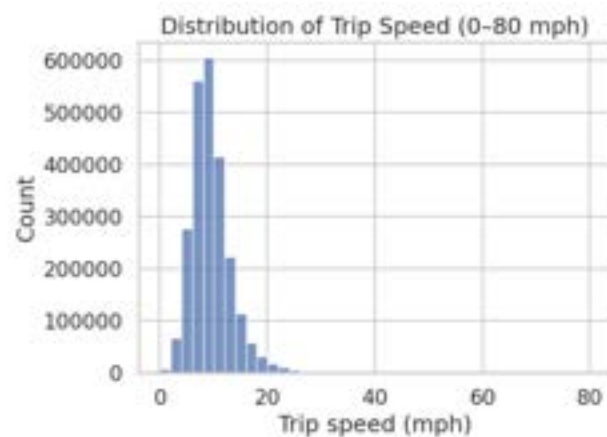
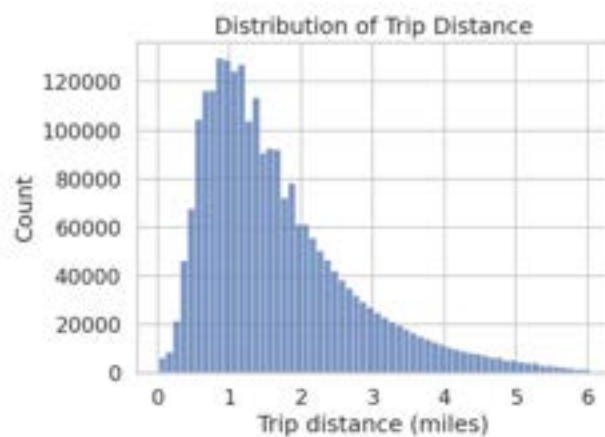
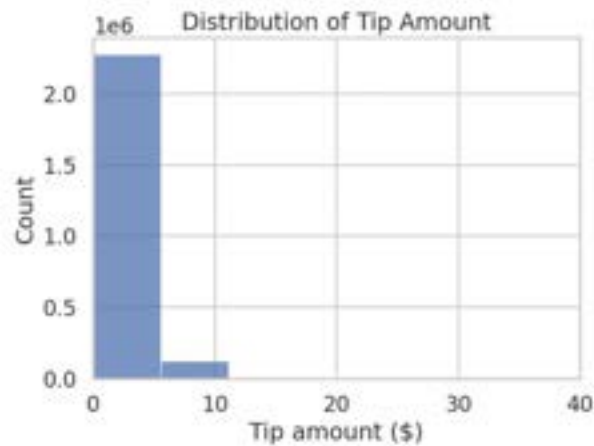
### 4. Exploratory Data Analysis

#### 4.1 Distributions of key variables

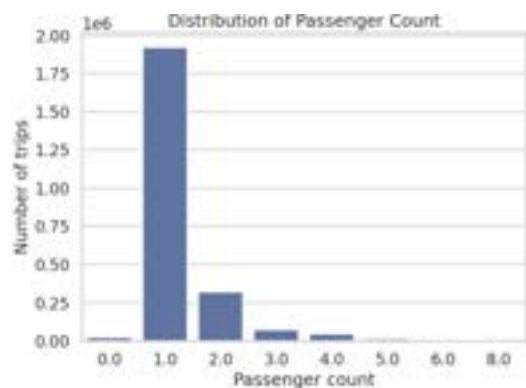


We first inspect summary statistics for the main numerical variables (`fare_amount`, `tip_amount`, `tip_percent`, `trip_distance`, `trip_hours`, `trip_speed`, `passenger_count`), followed by several histograms. The histogram of tip percentage (with the analysis restricted to tips below 50%) shows a right-skewed distribution with a clear concentration of trips between roughly 10–40%, and a tail of higher

percentages. A histogram of tip amount indicates that most tips are only a few dollars, with relatively few large tips.

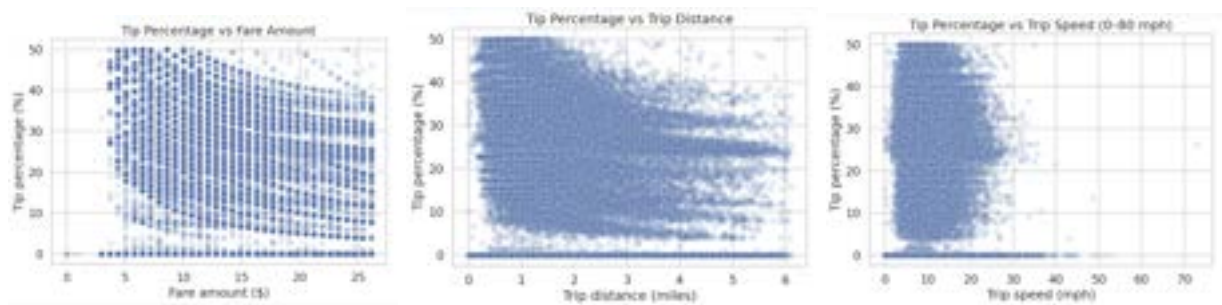


A 2×2 grid of histograms for tip amount, fare amount, trip distance, and trip speed shows that all of these variables are right-skewed: most rides are short and inexpensive, while long, high-fare trips are much less common. For trip speed, we restrict the plot to trips with speeds between 0 and 80 mph so that the x-axis reflects realistic urban driving speeds rather than being dominated by a small number of extreme values caused by very short recorded durations.

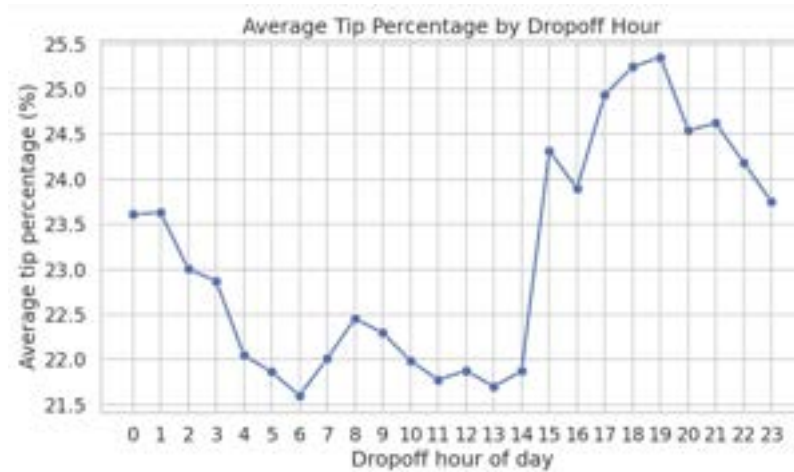


A count plot of passenger count reveals that the vast majority of trips involve a single passenger, with decreasing frequency for two or more passengers and very few rides with large groups.

## 4.2 Relationships between tip percentage and other variables

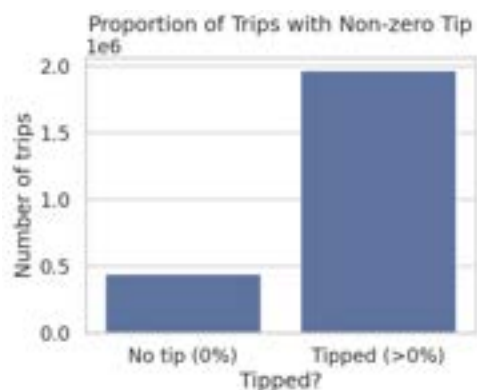


To explore relationships between tipping and other trip characteristics, we create scatterplots of tip percentage versus fare amount, trip distance, and trip speed, using a random sample of 100,000 trips for readability and restricting plotted tip percentages to the range 0–50%. The tip percentage versus fare amount plot shows a dense cloud with substantial variance at almost every fare level. There is a weak overall downward trend: on very low fares we sometimes see quite high tip percentages, whereas at higher fares the points tend to concentrate in the 10–40% range, but the relationship is far from deterministic. The tip percentage versus trip distance scatterplot looks similarly noisy: riders tip both generously and minimally on short and long trips, with only a slight tendency for very short rides to show more variability and occasionally lower tip percentages. When plotting tip percentage versus trip speed (with speeds capped at 80 mph in the visualization), we again observe a broad spread of tipping behavior across the full speed range and no strong visual pattern.



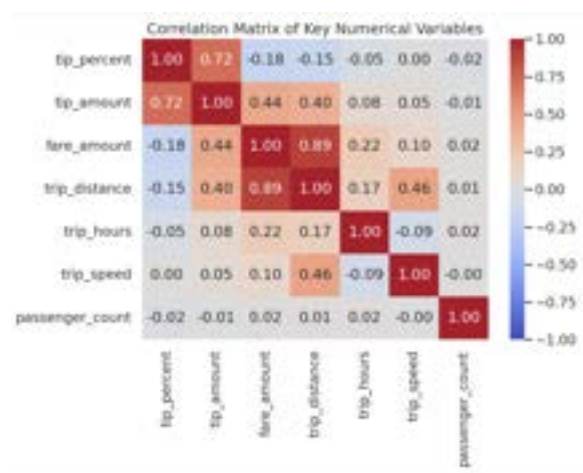
variability in tipping behavior.

We also examine time-of-day effects by plotting the average tip percentage as a function of dropoff hour. This line plot shows modest variation over the 24-hour cycle: some hours, often in the evening or late at night, exhibit slightly higher average tip percentages, while off-peak hours tend to be somewhat lower. However, these differences are relatively small compared with the overall



Finally, we summarize the binary tipped variable. A bar plot of tipped versus count shows that most trips in the cleaned dataset have a strictly positive tip percentage, but a non-trivial fraction still have  $\text{tip\_percent} = 0$ , corresponding to rides with no recorded tip. Normalized value counts provide the exact proportion of tipped versus non-tipped trips, which we use later when comparing continuous tip-percentage models with potential formulations that explicitly model whether a tip is given at all.

### 4.3 Correlations and noise



To quantify relationships among variables, we compute a correlation matrix over the main numerical features and visualize it with a heatmap. As expected, fare amount is correlated with trip distance and trip duration, reflecting the fare structure in which longer trips cost more. Tip amount is moderately correlated with fare amount and fare distance, since larger fares create room for larger absolute tips even when the percentage is similar. In contrast, tip percentage shows only weak correlations with distance, duration, speed, or passenger count, which is consistent with the

noisy scatterplots described above.

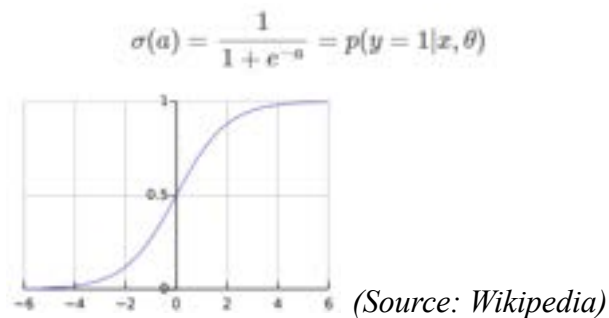
During exploration, we also observe a small number of extreme derived values in the raw data—for example, tip percentages above 1,000% or trip speeds above 10,000 mph. These typically arise when moderate numerators (tip or distance) are divided by very small denominators (very small fares or very short recorded durations). Our preprocessing mitigates much of this noise by removing non-positive fares and distances, filtering outliers in `trip_distance` and `fare_amount`, and excluding trips with tip percentage greater than or equal to 50%. For visualization, we further restrict some plots to plausible ranges (e.g., trip speeds between 0 and 80 mph) so that remaining extreme points do not distort the axes. Overall, the exploratory analysis indicates that, even after this cleaning and outlier filtering, the dataset remains noisy from the perspective of tip prediction: tipping behavior shows high variability and only weak linear relationships with the most obvious trip-level features, which motivates the use of more flexible, non-linear regression models in the subsequent analysis.

## 5. Methods

As part of this research, binary classification models to predict whether passengers tipped and regression models to predict tip amount as a percentage of fare were constructed. For classification, logistic regression, naive bayes and support vector classifier models were tested and for regression, linear, LASSO, ridge, elastic net and gradient boost regression were tested. Neural networks were also implemented for both tasks. The prediction variables used were drop-off hour, trip distance, trip speed, passenger count, fare amount and payment type and the response variables were tip percentage for the regression models and the boolean tip occurrence variable for the classification models. The cleaned data (after outliers and unrealistic data points were removed) was randomly split into a model training set containing 80% of the data points and a model testing set containing the remaining 20%.

### 5.1 Classification Models

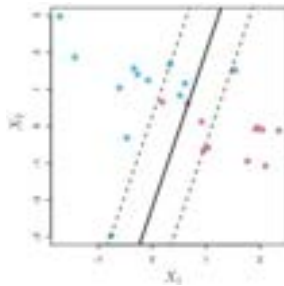
Of the three classification models built, logistic regression was the simplest. It only permits binary classification, but this was sufficient for the given classification task of predicting whether or not tipping occurred for a given set of parameters. The model utilizes simple linear regression (in which a linear relationship between the variables is developed) and then passes the results through a sigmoid function.



This function normalizes the data points between 0 and 1, thereby mapping it to its likelihood of being in class 1 as opposed to class 0. If this probability is greater than 0.5 (or different specified threshold), the data point is classified as the class corresponding to 1 and if it is less than 0.5, it is classified as that corresponding to 0. The second classification technique used was naive bayes. This applies Bayes' theorem to determine the probability of a specific class being predicted (A) from a given set of prediction variables (features) (B) using the likelihood of the prediction feature set being predicted using the given class:  $P(A|B) = P(B|A) * P(A) / P(B)$ . The “naive” assumption made here is that the prediction/feature variables are independent of each other. This is very much not the case for our dataset as many of the trip variables are correlated and codependent. Therefore, this presents a possible source of inaccuracy within our naive bayes model. The last model implemented was the support vector classifier model which performs classification using support vector machines. It works to identify a hyperplane (a decision



boundary such as a line, plane, or higher dimensional subspace) that best separates the data between the different classes.

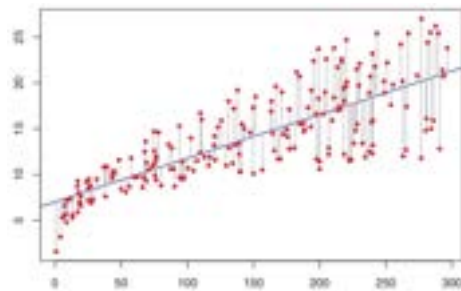


(Source: ISLP textbook)

The model attempts to maximize the margin which is the gap between the boundary and support vectors to better distinguish the classes. As it is able to work with higher dimensions, it is particularly effective at handling non-linear data. Surprisingly, all three classification models yielded almost identical accuracy when applied to the data.

## 5.2 Regression Models

Of the regression models, linear regression was the simplest, determining a linear relationship between the variables. The least squares method was used to minimize the residuals (errors) between the model predictions and actual data.



(Source: ISLP textbook)

LASSO, ridge and elastic net regression all make use of linear regression but with an added penalization feature incorporated into the models. In Least Absolute Shrinkage and Selection Operator (LASSO) regression, also known as L1 penalization, a penalty equal to the absolute value of the weight is added to the cost function. The model is represented mathematically by the following equation:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \alpha \sum_{i=1}^2 |\beta_i|$$

It is possible to remove variables from the model by setting their corresponding coefficients to zero (i.e. ignoring them completely), resulting in automatic feature selection and an overall sparser model. In ridge regression, also known as L2 penalization, the penalty added to the cost weight is equal to the square of the weight. It is represented mathematically by the following:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \alpha \sum_{i=1}^2 \beta_i^2$$

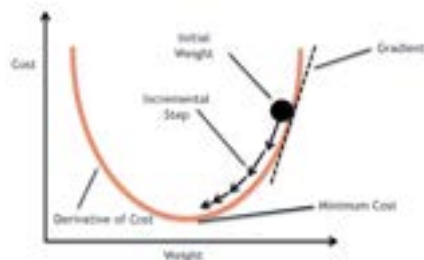
Consequently, it shrinks weights proportionally without removing any variables from the model. Both models are less sensitive to outliers and also reduce overfitting to the training data. Elastic



net regression, which was tested next, is a combination of LASSO and ridge regression which incorporates L1 and L2 penalizations, balanced by the alpha parameter as shown:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \alpha [L \sum_{i=1}^2 \beta_i^2 + (1 - L) \sum_{i=1}^2 |\beta_i|]$$

It is generally more stable and flexible than LASSO or ridge regression models alone as it can perform some feature selection and also handle multicollinearity. Lastly, gradient boost regression was implemented. In this model, decision trees (weak learners with accuracy slightly above that of random guessing) are generated sequentially. Each new tree will attempt to fill the gap of residuals (errors) collected from the previous trees. This technique uses gradient descent to identify the optimal direction for minimizing the loss function.

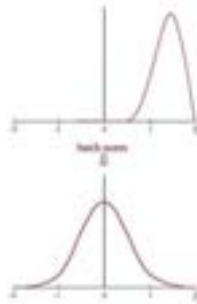


(Source: Vidhya Analytics)

The learning rate parameter of the model can be adjusted to reduce the possibility of the model overfitting or underfitting. Of the five regression models, this one was the most accurate/least prone to error.

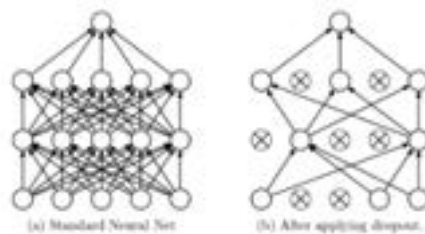
### 5.3 Neural Networks

Two similarly structured neural networks were implemented for classification and regression tasks. Both models initially take all nine input variables and process them using a dense (fully connected) layer with 64 nodes. The results from this layer are passed into a dense layer with 32 nodes and these results are in turn passed into a dense layer with 16 nodes. In both models, ReLU (Rectified Linear Unit: where positive values are mapped to themselves and negative values are mapped to zero) activation is used at every layer except the final output layer. In the regression model, these final 16 values are converted to a single output in the output layer with no activation, whereas in the classification model, the 16 values are converted to a single binary output using sigmoid activation (mapping to a probability between 0 and 1). The classification neural network is somewhat more sophisticated than the regression network as it incorporates batch normalization at the first layer and dropout at the second layer. Batch normalization assists in stabilizing activations by normalizing them within each mini-batch, ensuring it does not saturate at one point or explode. It can also reduce sensitivity to initializations.



(Source: Lecture Slides)

Dropout involves randomly removing a portion of neurons/nodes (in our case 20%) in each layer, with a different set removed (“dropped”) at each iteration. This reduces overfitting by ensuring that the network is not over-reliant on specific nodes.



(Source: Lecture Slides)

Together, these different layers and techniques work to enhance the quality of the neural network and improve its overall performance in both classification and regression tasks as desired.

## 6. Results (Metrics and Hyperparameters)

This section is split up between the results gathered for the classification methods as well as the regression methods. It will also cover the metrics used as well as the important hyperparameters used for each model, as they have a direct impact on what kind of results a model produces.

### 6.1 Classification Methods

#### *Metrics Used*

*Confusion Matrices:* When confusion matrices are used in our code to visualize accuracy, we also use a corresponding definition of accuracy:

$$accuracy = \frac{TP + TN}{TP + FN + FP + TN}$$

Where TP is the amount of true positives (correctly classified data), FN the amount of false negatives, FP the amount of false positives, and TN the amount of true negatives.

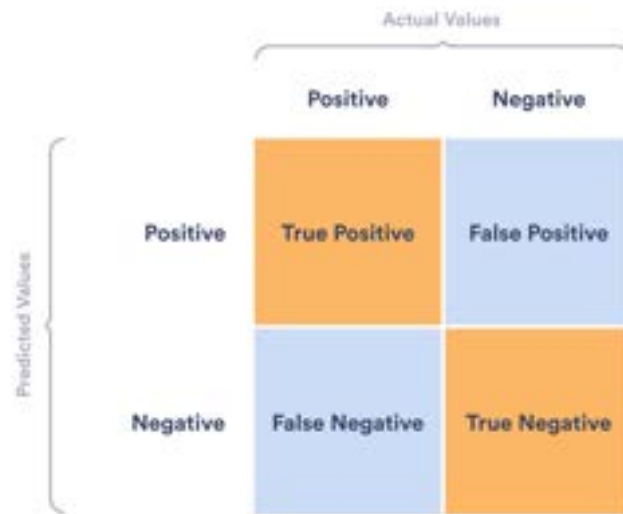


Figure: Confusion Matrix structure. Source: [Lory Seraydarian](#)

**AUROC (Area Under ROC):** While confusion matrices are useful for measuring the accuracy of classification models, another particularly useful metric is measuring the area under an ROC curve, or AUROC for short. Using the variables mentioned above and defining a true positive rate (all true positives over all positives) and a false positive rate (all false positives over all negatives) and plotting the true positive rate against the false positive rate at various thresholds (boundary between classes), an ROC curve can be produced:

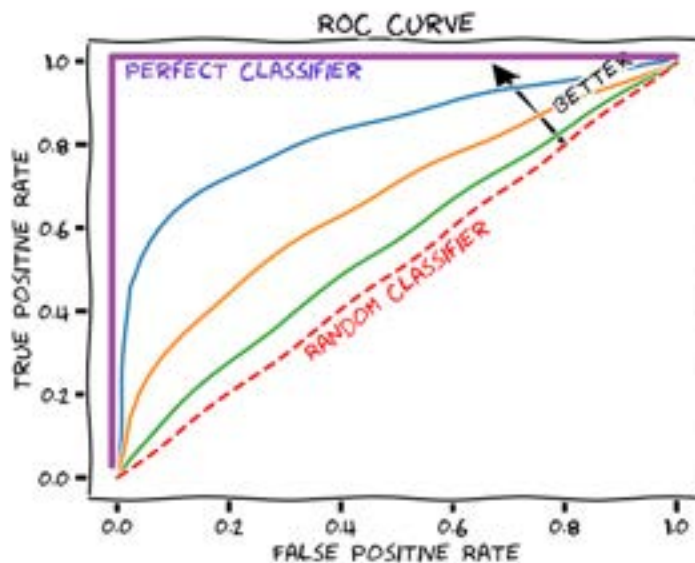


Figure: ROC curve example. Source: [Rachel Draelos](#)

- *Tensorflow Keras Accuracy Metric* (`tf.keras.metrics.Accuracy`): According to the TensorFlow documentation, “This metric creates two local variables, `total` and `count` that are used to compute the frequency with which `y_pred`(predicted value for `y`) matches `y_true`(the true values of `y`). This frequency is ultimately returned as a `binary accuracy`: an idempotent operation that simply divides `total` by `count`.”
- *Tensorflow Keras AUC Metric* (`tf.keras.metrics.AUC`): Method of choice of computing the AUROC, mentioned above. According to the TensorFlow documentation, “This class approximates AUCs using a Riemann sum. During the metric accumulation phrase, predictions are accumulated within predefined buckets by value. The AUC is then computed by interpolating per-bucket averages. These buckets define the evaluated operational points. This metric creates four local variables, `true_positives`, `true_negatives`, `false_positives` and `false_negatives` that are used to compute the AUC.”
- *Binary Cross Entropy Loss* (`tf.keras.losses.BinaryCrossentropy`): According to the TensorFlow documentation, “Computes the cross-entropy loss between true labels and predicted labels.”

## Models:

Each model has its metrics used, results, and important hyperparameters used. Assume that if a hyperparameter in a model is not mentioned that the default was used in our code.

### *Logistic Regression*

Metrics: Confusion Matrix

Results:

- Accuracy: 0.96
- Notes: Our baseline standard that we compare the following classification models to.

Hyperparameters:

- `fit_intercept = true`: Adds an extra variable in the equation to be optimized. It allows the decision boundary, which represents the baseline probability of the outcome in the data, to be shifted away from the origin (an example of such a boundary would be like 50%, half goes to Class A, and Class B. If this is on, it allows that baseline to shift higher or lower as needed for the data). In practice, it is always recommended to have this parameter be true – it's essential to making logistic regression applicable to different types of data.
- `random_state = 301`: Makes sure our results are reproducible.
- `max_iter = 100`: The maximum number of iterations LogReg will perform during the optimization process to find the best solution. The lower `max_iter` is, the faster but possibly less accurate the results are, the higher it is the slower but possibly more

accurate the results are, since the model was given more time to optimize its parameters.

### *Naive Bayes*

Metrics: Confusion Matrix

Results:

- Accuracy: 0.96
- Notes: Higher amount of false positives compared to logistic regression (126 vs 12)

Hyperparameters: N/A

### *Support Vector Machines (Linear Kernel):*

Metrics: Confusion Matrix

Results:

- Accuracy: 0.96
- Notes: Same accuracy as Logistic Regression and Naive Bayes. Used only linear kernel, since the model is only working with two classes: tipped or not tipped. As such, testing other kernels was unnecessary.

Hyperparameters:

- `penalty='l2'`: Employs an L2 penalty term for regularization. L2, or Ridge, regularization allows the model to prevent overfitting in the data.
- `loss='squared_hinge'`: Uses the “squared hinge” loss function, which is the square of the hinge loss.
- `C=10`:  $C$  corresponds to the inverse of the strength of our SVM’s regularization. Decreasing  $C$  allows a wider margin in our decision boundary and more margin violations (as in, points ending up on the wrong side of the margin), and increasing  $C$  fits the training data more tightly with fewer violations.
- `fit_intercept=True`: Same purpose as in Logistic Regression
- `random_state=301`: Same purpose as in Logistic Regression
- `max_iter=10000`: Same purpose as in Logistic Regression. Higher values for `max_iter` are preferred for SVM.

### *Neural Network*

Metrics: Tensorflow Keras Accuracy Metric, Tensorflow Keras AUC Metric, Binary Cross Entropy Loss

Results:

- Accuracy: 0.9643, AUC: 0.9270. Loss: 0.1437

Notes: Performed slightly better in terms of accuracy than the other models.

Hyperparameters:

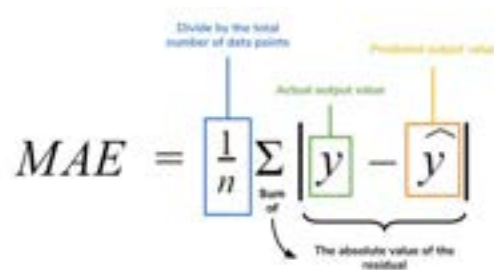
- `optimizer='adam'`: Used the ADAM optimizer due to it being suited for large datasets. Used to adjust learning rates during training.
- `epochs=20`: Ran for 20 epochs aka the amount of times the model runs.

- `batch_size=1024`: Controls how the training data is divided up into batches, helping with training speed and accuracy.
- `validation_split=0.1`: Validation split of 0.1 means that 10% of the training data will be set aside as a validation set to evaluate the model's performance during training.

## 6.2 Regression Methods

### *Metrics Used*

*Mean Absolute Error (MAE)*: Measures the average absolute differences between predicted values and actual values.



The diagram shows the MAE formula:  $MAE = \frac{1}{n} \sum |y - \hat{y}|$ . Annotations include:
 

- A blue box around  $\frac{1}{n}$  with a label "Divide by the total number of data points".
- A green box around  $y$  with a label "Actual output value".
- An orange box around  $\hat{y}$  with a label "Predicted output value".
- A bracket under the absolute value term  $|y - \hat{y}|$  with a label "The absolute value of the residual".
- A label "Sum of" pointing to the summation symbol  $\sum$ .

Figure: MAE formula. Source: [Dataquest](#)

### **Models:**

Each model has its metrics used, results, and important hyperparameters used. Assume that if a hyperparameter in a model is not mentioned that the default was used in our code.

#### *Linear Regression*

Metrics: MAE

Results: Error: 8.0033% avg error

Notes: Our baseline standard that we compare the following classification models to.

Hyperparameters: N/A

#### *LASSO Regression*

Metrics: MAE

Results: Error: 8.0033% avg error

Notes: Same as linear regression but with L1 regression. Performed the same as linear regression without L1.

Hyperparameters:

- `alphas = [0.0001, 0.001, 0.01, 0.1, 1.0, 10.0, 100, 1000]`: Alphas are the

set of penalty terms for the L1 regularizer.

### *RIDGE Regression*

Metrics: MAE

Results: Error: 8.0033% avg error

Notes: Same as linear regression but with L2 regression. Performed the same as linear regression without L2.

Hyperparameters:

- `alphas = [0.001, 0.01, 0.1, 1.0, 10.0, 100, 1000]`: Same as LASSO, alphas are the set of penalty terms for the L2 regularizer.

### *ElasticNetCV*

Metrics: MAE

Results: Error: 8.0036% avg error

Notes: Slightly worse than previous methods.

Hyperparameters:

- `alphas = np.logspace(-4, 4, 10)`: Similar to LASSO and RIDGE, the alphas are tested along the regularization path and used for each `l1_ratio` (compromise between l1 and l2 penalization chosen by cross validation).
- `random_state = 301`: Makes sure our results are reproducible.
- `max_iters = 1000`: The maximum number of iterations ElasticNetCV will perform during the optimization process to find the best solution. The lower `max_iter` is, the faster but possibly less accurate the results are, the higher it is the slower but possibly more accurate the results are, since the model was given more time to optimize its parameters

### *Gradient Boosting Regressor (GBR)*

Metrics: MAE

Results: Error: 6.2319%

Notes: Performs significantly better than previous models.

Hyperparameters:

- `learning_rate=0.3`: Controls how much each tree will contribute to the output of the predicted value.
- `n_estimators=50`: Numbers of regression estimators, or trees.
- `max_depth=10`: Maximum depth of the individual regression estimators of the model

### *Neural Network*

Metrics: MAE

Results: 6.2420%

Notes: Performs around the same as GBR.

Hyperparameters:



- `optimizer='adam'`: Used the ADAM optimizer due to it being suited for large datasets. Used to adjust learning rates during training.
- `epochs=20`: Ran for 20 epochs aka the amount of times the model runs.
- `batch_size=1024`: Controls how the training data is divided up into batches, helping with training speed and accuracy.
- `validation_split=0.1`: Validation split of 0.1 means that 10% of the training data will be set aside as a validation set to evaluate the model's performance during training.

## 7. Conclusion

In this project, we applied various classification models, including logistic regression, Naive Bayes, support vector machines, and a neural network, to predict whether New York City yellow taxi trips receive a tip. All four models achieved similar performance, with accuracies of approximately 0.96. While this is a strong result, it is important to interpret it in context, as the dataset is imbalanced, with roughly four out of five trips receiving a tip. As a result, accuracy alone can be a somewhat misleading metric. Although the neural network performed slightly better than the other classifiers, the improvement was relatively small. We also observed meaningful differences in errors across models; for example, Naive Bayes produced relatively more false positives, while logistic regression resulted in more false negatives, highlighting the tradeoff between precision and recall when deciding which model to utilize for this task. For the regression task of predicting tip percentage, we evaluated several models, including linear regression, ridge regression, lasso regression, Elastic Net, gradient boosting, and a neural network. Most models achieved a mean absolute error of around 8%, while the gradient boosting regressor and neural network performed best, with MAE values of approximately 6%. This suggests that, on average, these models' predictions differ from the true tip percentage by about six percentage points, which is a reasonably strong result given the inherent variability and noise in tipping behavior. Overall, these results demonstrate both the difficulty of predicting something that is inherently based on human decision-making in real-world settings, as well as the potential of using trip-level information to model tipping behavior in order to achieve meaningful results.

## 8. Future Work

There are several ways in which this project could be expanded upon. First, in this project, all of our training and testing was performed using data from January 2025. While this dataset is already very large, with over two million observations after filtering, additional data is available going back through 2009. By incorporating data on trips from prior years, this could provide insight into whether tipping behavior has changed over time. If this is the case, then the year of the trip itself could become a meaningful input feature for our classification and regression tasks. In addition, expanding the timeframe over which our data was collected would allow us to draw stronger conclusions regarding how well our models could generalize to unseen months or years.

Secondly, it is important to note that our analysis focused on Yellow taxi trips, but additional datasets, including Green taxi trips, are available from the NYC Taxi & Limousine Commission as well. Green taxis are designed to focus more on underserved areas in the outer boroughs, and are prohibited from taking passengers from parts of Manhattan due to excess supply. This Green taxi dataset could be included to study how tipping behavior varies when comparing datasets that prioritize different geographic areas. In addition, there is also data available on trips that were taken with for-hire services, which include data provided by companies such as Uber and Lyft. These could be especially interesting to analyze because these companies often use tactics such as surge pricing during busier times, and these higher prices may impact how much passengers would be willing to tip.

Another interesting idea could be to incorporate external datasets. For example, it's possible that tipping patterns are different on holidays, so incorporating an indicator variable representing whether or not a trip was taken on a holiday may improve accuracy. Similarly, it's possible that major events, such as baseball games or concerts, may influence traffic patterns, and thus may also have an effect on passenger tipping behavior. Thus, including an indicator variable representing whether a user was taking a trip to an event (e.g., their dropoff location was Yankee Stadium at a time that matches a Yankees game) could provide useful information. A third idea for an external dataset is a historical weather dataset containing information about temperature and precipitation. It is possible that weather conditions may have predictive value in this context, as weather could affect a passenger's trip experience and their willingness to tip.

One final point to note is that although our dataset includes pickup and dropoff zone IDs, we did not directly use them as model inputs. These IDs correspond to specific zones and are not tied to latitude and longitude coordinates, meaning that they aren't distributed in a geographically logical way and cannot be treated as continuous variables. If we wanted to use these zone IDs as an input to our regression or classification models, it would not have made sense to use the numbers directly. One alternative could be to one-hot encode them, but that would not be practical either, since there are hundreds of zone IDs. A more appropriate option could be to one-hot encode the borough (e.g., Manhattan, Brooklyn, etc.) associated with the ID. This could be done using a taxi-zone csv lookup table, which is linked on the TLC Trip Record Data page. This would likely require performing a join query between this taxi-zone csv and the filtered dataset from the parquet file we used. It would be interesting to do this if we were to work on this project further, or even implement an unsupervised learning algorithm to determine a way to logically cluster location zones. This would allow us to see whether different geographic areas have different trends regarding tipping, and could potentially improve predictive accuracy even further.

## References

- Boateng, D. (n.d.). *Predicting taxi gratuities in New York City* [Computer software]. GitHub. Retrieved December 15, 2025, from [https://github.com/Dan-Boat/predict\\_taxi\\_tip](https://github.com/Dan-Boat/predict_taxi_tip)
- Jain, S., See, A., & Shandilya, A. (2015). *Predicting taxi tip-rates in NYC* [Unpublished course project]. Department of Computer Science and Engineering, University of California, San Diego.
- New York City Taxi & Limousine Commission. (2025a). *TLC trip record data* [Data set]. NYC.gov. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- New York City Taxi & Limousine Commission. (2025b). *Data dictionary – Yellow taxi trip records* [Data dictionary]. NYC.gov. [https://www.nyc.gov/assets/tlc/downloads/pdf/data\\_dictionary\\_trip\\_records\\_yellow.pdf](https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf)
- Rathore, B., Sengupta, P., Biswas, B., & Kumar, A. (2024). Predicting the price of taxicabs using artificial intelligence: A hybrid approach based on clustering and ordinal regression models. *Transportation Research Part E: Logistics and Transportation Review*, 185, 103530. <https://doi.org/10.1016/j.tre.2024.103530>
- Wang, H. (2024). Yellow taxi demand prediction for New York City based on VMD-SSA-LSTM. In *Proceedings of the 1st International Conference on Data Science and Engineering (ICDSE)* (pp. 434–440). SciTePress. <https://doi.org/10.5220/0012810400004547>